SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Experiments

COURSE NAME: DEEP LEARNING **COURSE CODE: MLA0408**

FACULTY : Dr. D.SUDHAGAR

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

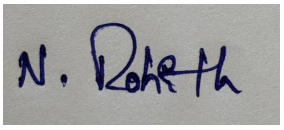
Duration: 30 minutes

Marks: 50

Code of Conduct:

I **NALIPI REDDY ROHITH/192425115** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



QUESTIONS:10 **Total Marks:50**

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a)Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

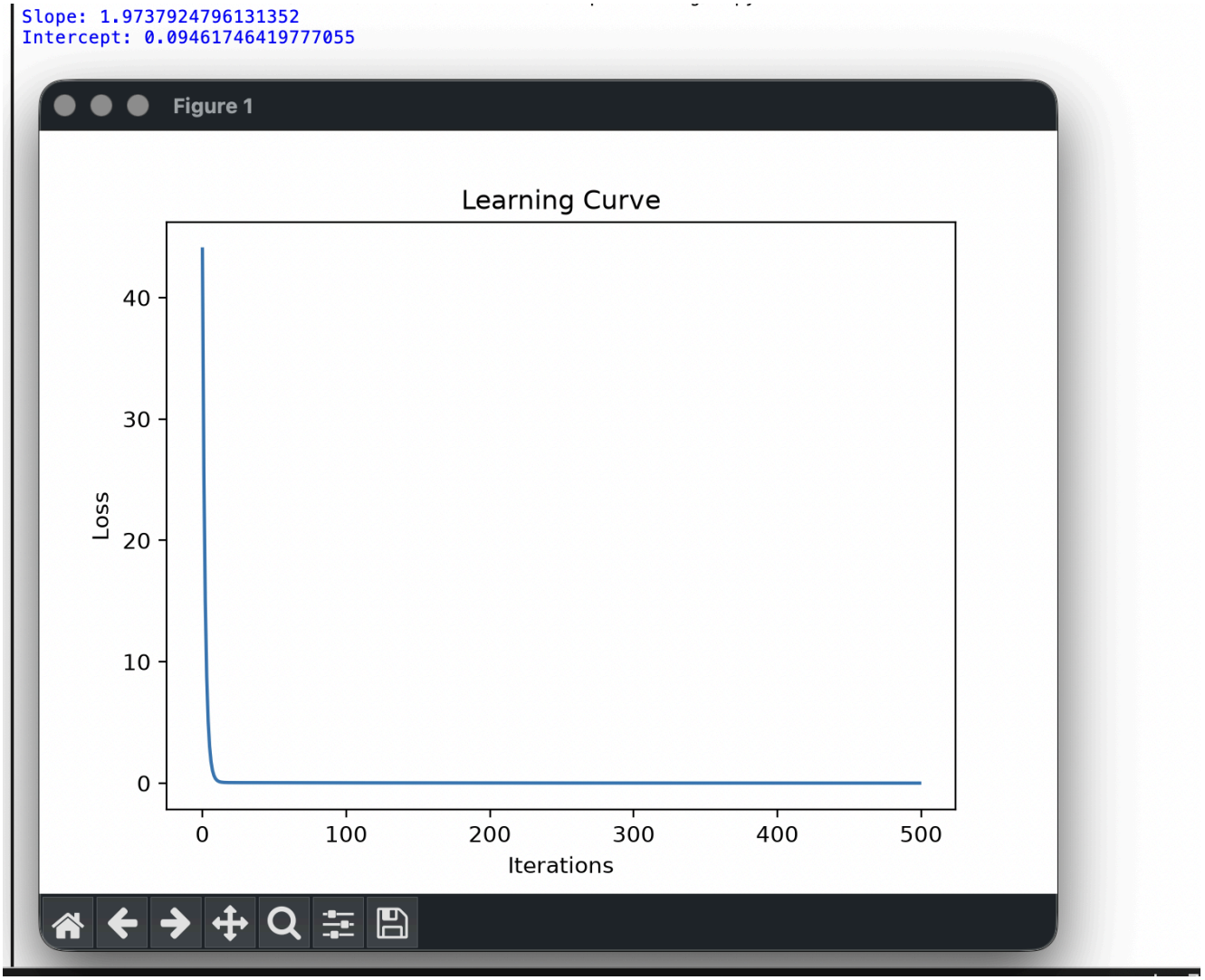
Experiments

1.Learning Algorithms

```
import numpy as np

import matplotlib.pyplot as plt


X = np.array([1,2,3,4,5], dtype=float)

y = np.array([2,4,6,8,10], dtype=float)


m = 0

b = 0

lr = 0.01

epochs = 500


loss = []


n = len(X)


for i in range(epochs):

    y_pred = m*X + b

    error = y_pred - y


    dm = (2/n) * np.sum(error * X)

    db = (2/n) * np.sum(error)


    m -= lr * dm

    b -= lr * db
```

```
    loss.append(np.mean(error**2))


print("Slope:", m)

print("Intercept:", b)


plt.plot(loss)

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.title("Learning Curve")

plt.show()
```

```
Slope: 1.9737924796131352
Intercept: 0.09461746419777055
```



2.Maximum Likelihood Estimation (MLE)

```
import numpy as np


np.random.seed(0)


true_mean = 5

true_std = 2


data = np.random.normal(true_mean, true_std, 1000)


mle_mean = np.mean(data)

mle_variance = np.mean((data - mle_mean)**2)


print("Actual Mean:", true_mean)

print("Estimated Mean:", mle_mean)


print("Actual Variance:", true_std**2)

print("Estimated Variance:", mle_variance)
```

```
============== RESTART: /Users/rohith/Documents/deep lea
Actual Mean: 5
Estimated Mean: 4.909486585019609
Actual Variance: 4
Estimated Variance: 3.896937825248617 3
>>
```

3.Building Machine Learning Algorithms

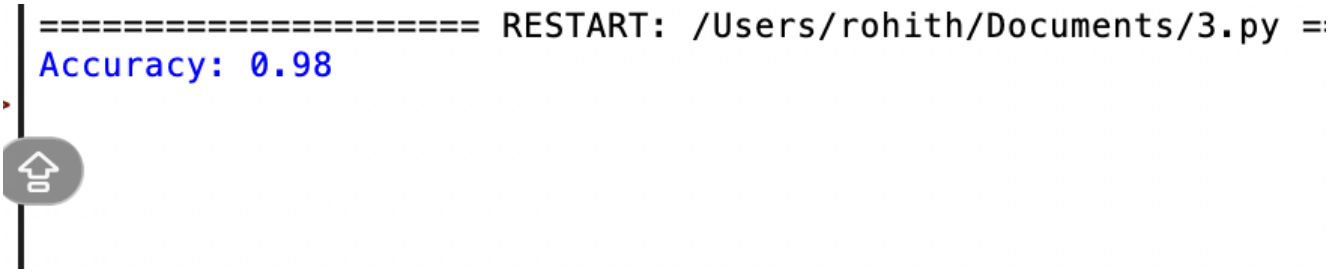
```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix


data = load_iris()


X_train,X_test,y_train,y_test = train_test_split(

    data.data,data.target,test_size=0.2,random_state=42)


scaler = StandardScaler()


X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


model = LogisticRegression()


model.fit(X_train,y_train)


pred = model.predict(X_test)


print("Accuracy:",accuracy_score(y_test,pred))

print(confusion_matrix(y_test,pred))
```

```
========== RESTART: /Users/rohith/Documents/3.py ==========
Accuracy: 1.0
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
>
```

4.Neural Networks

```
from sklearn.datasets import make_moons

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier


X,y = make_moons(n_samples=300,noise=0.2,random_state=0)


X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)


model=MLPClassifier(hidden_layer_sizes=(10,),max_iter=1000)


model.fit(X_train,y_train)


print("Accuracy:",model.score(X_test,y_test))
```



5.Artificial Neurons

```
import numpy as np


x = np.array([1,2,3])


w = np.array([0.2,0.4,0.6])


b = 0.5


z = np.dot(x,w)+b
```

```
sigmoid = 1/(1+np.exp(-z))


relu = max(0,z)


print("Neuron Output:",z)

print("Sigmoid:",sigmoid)

print("ReLU:",relu)
```

```
====================== RESTART: /Users/rohith/Documents/3.py ============
Neuron Output: 3.3
Sigmoid: 0.9644288107273639
ReLU: 3.3
```

6.Perceptron and Multilayer Perceptron (MLP)

a)

```
from sklearn.datasets import make_classification

from sklearn.linear_model import Perceptron


X,y = make_classification(

    n_samples=100,

    n_features=2,

    n_redundant=0,

    n_clusters_per_class=1,

    random_state=0)


model = Perceptron()


model.fit(X,y)


print("Accuracy:",model.score(X,y))
```

```
========== RESTART: /Users/rohith/Documents/3.py ===
Accuracy: 0.96
```

b)

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier


iris = load_iris()


X_train,X_test,y_train,y_test = train_test_split(

iris.data,iris.target,test_size=0.2)


mlp = MLPClassifier(hidden_layer_sizes=(20,),max_iter=1000)


mlp.fit(X_train,y_train)


print("MLP Accuracy:",mlp.score(X_test,y_test))
```

```
========== RESTART: /Users/rohith/Documents/3.py ==========
MLP Accuracy: 0.9666666666666667
```

7.Forward Propagation and Backpropagation Algorithm

a)

```
import numpy as np


x = np.array([1,2])


w = np.array([[0.5,0.3],

        [0.4,0.7]])
```

```
hidden = np.dot(x,w)

output = 1/(1+np.exp(-hidden))

print(output)
```

```
====================== RESTART: /Users/rohith/Documents/3.py ====================
[0.78583498 0.84553473]
```

b)import numpy as np

```
# Input and Target
x = 1.0
target = 1.0

# Initial Weight
w = 0.5

# Learning Rate
lr = 0.1

# Forward Propagation
net = x * w
output = 1 / (1 + np.exp(-net))

# Error
error = target - output

# Backpropagation
gradient = error * output * (1 - output) * x
```

```
# Weight Update

w = w + lr * gradient


print("Output:", output)

print("Error:", error)

print("Gradient:", gradient)

print("Updated Weight:", w)
```

```
========== RESTART: /Users/rohith/Documents/3.py ==========
Output: 0.6224593312018546
Error: 0.3775406687981454
Gradient: 0.08872345867463687
Updated Weight: 0.5088723458674637
>
```

8.Gradient Descent and Stochastic Gradient Descent (SGD)

a)

```
import numpy as np


x = 5


lr = 0.1


for i in range(20):
    grad = 2*x
    x = x-lr*grad
    print(i,x)
```

```
========================================================== RESTART: /U
0 4.0
1 3.2
2 2.56
3 2.048
4 1.6384
5 1.31072
6 1.0485760000000002
7 0.8388608000000002
8 0.6710886400000001
9 0.5368709120000001
10 0.4294967296000001
11 0.3435973836800001
12 0.27487790694400005
13 0.21990232555520003
14 0.17592186044416003
15 0.140737488355328
16 0.11258999068426241
17 0.09007199254740993
18 0.07205759403792794
19 0.057646075230342354
```

b)from sklearn.linear_model import SGDRegressor

import numpy as np

Training Data

X = np.array([[1], [2], [3], [4], [5]])

y = np.array([2, 4, 6, 8, 10])

SGD Model

model = SGDRegressor(

    max_iter=5000,

    tol=1e-3,

    learning_rate='constant',

    eta0=0.01,

    random_state=42

)

Train

```
model.fit(X, y)

# Predict
prediction = model.predict([[6]])

print("Prediction for x = 6:", prediction[0])
```

```
========== RESTART: /Us
Prediction for x = 6: 11.710737553946032
```

9.Mini-Batch Gradient Descent

```
from sklearn.neural_network import MLPRegressor
import numpy as np

X=np.random.rand(100,2)

y=X[:,0]+X[:,1]

model=MLPRegressor(batch_size=16,max_iter=500)

model.fit(X,y)

print(model.score(X,y))
```

```
========== RESTART: /Users/rohith/Documents/3.py ==========
0.9991905092658104
```

10. Curse of Dimensionality

```
import numpy as np
```

```
for d in [2,10,50,100,500]:
    X=np.random.rand(100,d)

    dist=[]

    for i in range(99):
        dist.append(np.linalg.norm(X[i]-X[i+1]))

    print("Dimension:",d)
    print("Average Distance:",np.mean(dist))
```

```
========== RESTART: /Users/rohith/Documents/3.py ==========
Dimension: 2
Average Distance: 0.4983829646423855
Dimension: 10
Average Distance: 1.2819156313269073
Dimension: 50
Average Distance: 2.8530727503813123
Dimension: 100
Average Distance: 4.020557976304174
Dimension: 500
Average Distance: 9.151235758802109
```